

深圳大学实验报告

课程名称: 基于 Web 的编程

实验项目名称: JS 综合实验

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: BASKER GEORGE

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 05 月 20 日至 07 月 1 日

实验报告提交时间: 2026 年 6 月 16 日星期二

教务处制

实验目的与要求:

目的: 能够使用 JS, 构思并设计、实现较完整的用户注册、登录等功能模块。

要求: 1、设计实现自己网站的用户注册和登录模块。

2、要求注册和登录模块用户体验良好, 做好安全防护方面的设计。

方法、步骤:

要完成本实验, 依据实验要求进行分解, 需要完成的实验步骤是:

1. 完成上机实验, 包括
 - 1.1 数字运算和循环
 - 1.2 偶数平方和
 - 1.3 动态计数器
 - 1.4 商品清单生成器
 - 1.5 学习主页优化
 - 1.6 星级评分
 - 1.7 待办事项
 - 1.8 学习主页再次优化
 - 1.9 英雄快跑游戏
 - 1.10 拖拽排序
 - 1.11 通讯录管理
 - 1.12 WebWorker
 - 1.13 用户注册
2. 设计并实现某主题网站的账户模块, 包括
 - 2.1 构思、设想
 - 2.2 设计实现及安全优化
 - 2.3 效果呈现
 - 2.4 思考小结

特别注意:

1. 不要贴大篇幅的源码, 连续代码行不超过 20 行。
2. 不要贴大体积的图片, 最后提交的文档不能超过 20M。
3. 作业提交: 提交 doc 或 pdf 文件任一即可, 需提交源代码;
4. 特别注意: 提交文件不要打包, 不要超过 20M (**高分辨率截图容易导致一个文件几百 M, 特别注意不要出现这种情况**)

实验过程及内容:

1. 上机实验完成情况:

- 1.1 数字运算和循环

- 1.1.1 实验过程 (包括是否遇到问题及如何解决)

本实验通过 JavaScript 实现基本的数字运算和循环操作。首先使用 var 关键字定义一个空数组 numbers, 然后使用 for 循环生成 1 到 10 的数字, 并将每个数字的平方值通过 push 方法添加到数组中。接着编写 calculateSum 函数, 该函数通过遍历数组累加每个元素来计算总和。最后使用 console.log 输出数组和总和。

遇到的问题: 初始编写时对 var 和 let 的作用域差异理解不够深入, 但本实验要求使用 var, 因此按照要求完成。另外计算总和的函数需要考虑空数组的情况, 初始值为 0 可以很好地处

理边界条件。

1.1.2 实验结果

在浏览器中打开控制台，成功在控制台输出数组[1,4,9,16,25,36,49,64,81,100]和总和 385。



1.2 偶数平方和

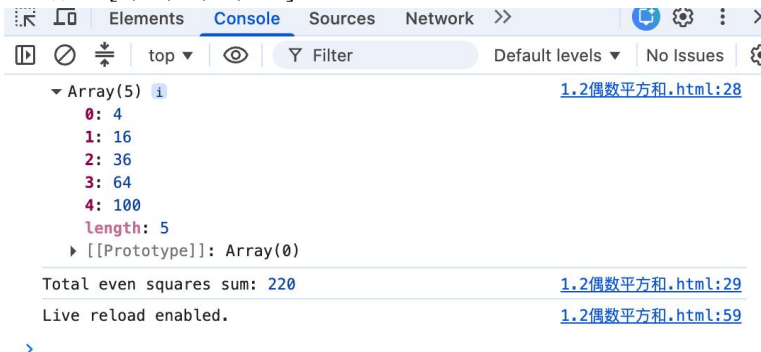
1.2.1 实验过程（包括是否遇到问题及如何解决）

本实验实现了偶数判断和平方计算功能。首先编写 `isEven(num)` 函数，使用取模运算符判断数字是否为偶数。然后实现 `square(num)` 函数返回数字的平方。接着使用数组方法链式调用：`filter(isEven)` 过滤出偶数、`map(square)` 计算平方、`reduce` 累加求和。这种函数式编程风格使代码简洁且可读性强。

遇到的问题：`reduce` 方法的使用需要注意初始值的设置，如果不传入第二个参数 0 作为初始值，在数组为空时可能会出错。另外需要确保 `filter` 和 `map` 中使用的是函数引用而非调用结果。

1.2.2 实验结果

成功输出偶数平方数组[4,16,36,64,100]和总和 220。



1.3 动态计数器

1.3.1 实验过程（包括是否遇到问题及如何解决）

本实验实现了一个带范围限制的动态计数器。页面包含计数器显示区域和加一、减一两个按钮。通过为按钮添加 `click` 事件监听器，在点击时更新计数器值。计数器值限制在 0 到 10 之间，超出范围时显示红色提示信息。

遇到的问题：需要处理边界条件——当计数器为 0 时点击减一按钮不能变为负数，为 10 时点击加一按钮不能超过 10。通过在事件处理函数中添加条件判断来解决。另外需要注意每次合法点击后清除之前的错误提示信息。

1.3.2 实验结果

当计数器<0 时候：



当计数器>10 的时候:



当计数器在 0~10 时:



1.4 商品清单生成器

1.4.1 实验过程 (包括是否遇到问题及如何解决)

本实验实现了商品清单的生成和显示。定义商品数组包含 5 个商品 (苹果、香蕉、葡萄、西

瓜、菠萝)，每个商品有 name 和 price 属性。遍历数组创建 li 元素，使用 textContent 设置显示文本格式为'商品名 - \$价格'。根据价格区间设置商品颜色：低于 50 为绿色 (low-price 类)、50-100 为蓝色 (mid-price 类)、高于 100 为红色 (high-price 类)，同时累加计算总价。
遇到的问题：需要确保价格区间的划分无遗漏，使用 if-else if-else 结构覆盖所有情况。另外 textContent 的格式需要严格按照' - \$'样式，不能有空格差异。

1.4.2 实验结果

成功生成商品清单，每个商品根据价格显示对应颜色，总价正确计算并显示为'总价: \$300'。5 个商品分别呈现绿色 (苹果\$30、香蕉\$20)、蓝色 (葡萄\$60、菠萝\$80) 和红色 (西瓜\$110)

商品清单

- 苹果 – \$30
- 香蕉 – \$20
- 葡萄 – \$60
- 西瓜 – \$110
- 菠萝 – \$80

总价: \$300

1.5 学习主页优化

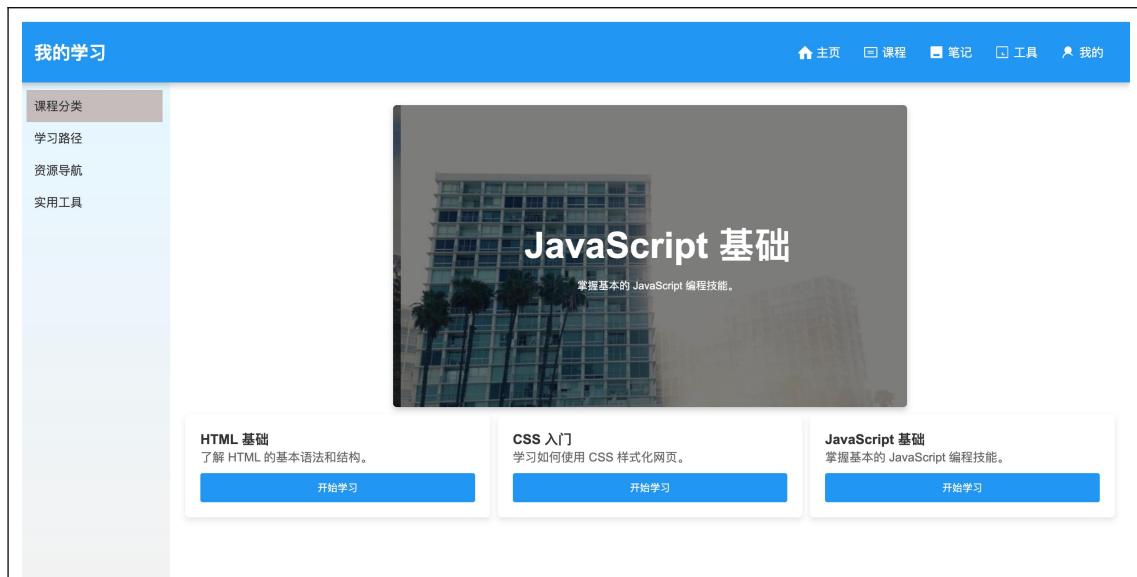
1.5.1 实验过程 (包括是否遇到问题及如何解决)

在提供的学习主页模板基础上进行了以下优化：

- (1) 添加网站图标 (favicon)，使用 SVG 内联方式设置 emoji 图标；
- (2) 为轮播图的三张卡片添加背景渐变叠加背景图片，使用 linear-gradient 叠加 picsum 图片实现暗色遮罩效果；
- (3) 完善 CSS 中的背景位置、尺寸等设置；
- (4) 在导航菜单中嵌入 SVG 图标，分别为主页、课程、笔记、工具和我的页面设计对应的内联 SVG 图标；
- (5) 设置 SVG 图标的样式类.icon 和悬停效果。

遇到的问题：SVG 图标的路径需要精确匹配，初始尝试时部分图标显示不正常，通过查阅 SVG path 规范修正。轮播背景图的渐变叠加需要注意 linear-gradient 和 url 的顺序。

1.5.2 实验结果



1.6 星级评分

1.6.1 实验过程（包括是否遇到问题及如何解决）

本实验实现了一个星级评分系统，支持 0.5 星到 5 星的半星评分。核心实现包括：

- (1) 使用 CSS 绝对定位的半星和全星高亮层，通过控制 `display` 属性实现显示/隐藏；
- (2) `calculateHoverRating` 函数通过 `getBoundingClientRect` 计算鼠标在星星中的相对位置，判断选择了整星还是半星；
- (3) `highlightStars` 函数遍历所有星星，根据评分值设置整星或半星高亮；
- (4) 鼠标悬停时临时显示高亮，移开后恢复当前评分；
- (5) 点击星星设置评分，重置按钮清零。

遇到的问题：半星判断需要精确计算鼠标在星星元素内的水平偏移量，使用 `event.clientX` 减去星星左边界位置，与星星宽度一半比较来判断半星。另外需要确保 `highlightStars` 中比较逻辑正确，特别是半星时 `starValue` 等于 `rating+0.5` 的情况

1.6.2 实验结果



当前评分=3.5星

重置评分

1.7 待办事项

1.7.1 实验过程（包括是否遇到问题及如何解决）

本实验实现了任务清单 (Todo List) 功能，包括添加任务、删除任务、任务状态切换以及清空全部/已完成任务。添加任务时检查输入是否为空，为空则在控制台输出提示；不为空则创建 li 元素，包含任务名称的 span 和删除按钮。点击 span 可切换删除线样式 (line-through)，实现任务完成状态标记。删除按钮移除对应任务项。清空全部任务直接清空 ul 的 innerHTML，清空已完成任务遍历所有 li 检查 span 的 textDecoration 样式。

遇到的问题：清空已完成任务时需要从后往前遍历 (i--)，因为删除元素会改变 DOM 节点列表。如果从前往后遍历会导致跳过部分元素。span 的 textDecoration 需要通过 style 属性读取，初始没有设置时为 empty string。

1.7.2 实验结果

添加任务：

任务清单

web课程期末 6.24

添加任务

web大作业 7.1

删除

清空全部任务

清空已完成任务

任务清单

输入新任务

添加任务

web大作业 7.1

删除

web课程期末 6.24

删除

清空全部任务

清空已完成任务

当任务完成时，点击该任务：

任务清单

输入新任务

添加任务

web课程期末 6.24

删除

web大作业 7.1

删除

清空全部任务

清空已完成任务

清空已完成任务：

任务清单

输入新任务

添加任务

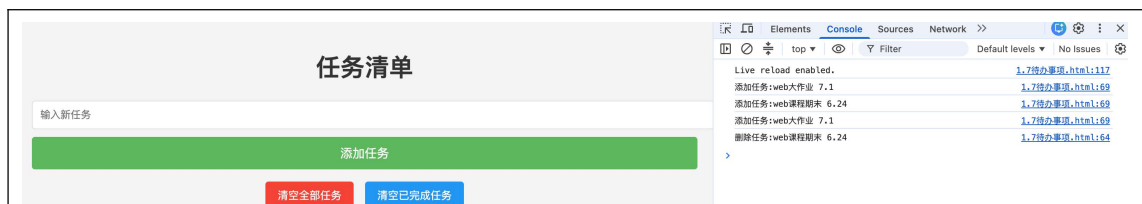
web课程期末 6.24

删除

清空全部任务

清空已完成任务

控制台正确打印信息：



1.8 学习主页再次优化

1.8.1 实验过程（包括是否遇到问题及如何解决）

在实验 1.5 学习主页优化的基础上进行了进一步优化：

- (1) 将所有样式从外部 CSS 引用改为内联样式表，包括完整的布局、配色、导航、侧边栏、卡片等样式；
- (2) 使用 CSS Grid 布局重构课程模块区域 (module-items)，使用 `repeat(auto-fit, minmax(250px, 1fr))` 实现响应式网格；
- (3) 为课程卡片添加悬停上浮动效 (`translateY(-5px)`) 和过渡动画；
- (4) 添加 JavaScript 轮播图自动播放功能，使用 `setInterval` 每 3 秒切换一张，通过 `transform: translateX` 实现平滑过渡；
- (5) 重新设计配色方案，使用渐变色 (`linear-gradient`) 作为导航栏和轮播图背景。

遇到的问题：轮播图的 `translateX` 计算需要精确匹配 `carousel-item` 的 `min-width: 100%`，使用 `currentIndex * 100 + '%'` 计算偏移量。卡片网格布局需要处理不同屏幕尺寸的适配。

1.8.2 实验结果

成功实现学习主页的全面优化，包含响应式网格布局、自动轮播图、卡片悬停动效和现代化配色方案。页面在功能和视觉上均达到更好的用户体验。



1.9 英雄快跑游戏

1.9.1 实验过程（包括是否遇到问题及如何解决）

本实验实现了一个完整的 Canvas 小游戏——英雄快跑。游戏机制是控制角色跳跃躲避移动的坑洞。核心实现包括：

- (1) 使用 `requestAnimationFrame` 驱动游戏主循环，实现流畅的 60fps 动画；
- (2) 角色物理系统——重力加速度 (`gravity=1`)、跳跃初速度 (`jumpStrength=-15`)，每帧更新位置和速度，检测着地状态；
- (3) 坑洞系统——初始生成 5 个随机宽度的坑洞 (30-50px)，随游戏进行向左移动，超出画布后移除并动态生成新坑洞；

- (4) 碰撞检测——判断角色与坑洞的矩形重叠；
- (5) 计时器和 localStorage 最佳成绩持久化。

遇到的问题：碰撞检测需要判断角色底部是否接触到地面 ($y+height \geq 270$)，且角色水平范围与坑洞重叠。requestAnimationFrame 的递归调用需要在 gameOver 时通过 cancelAnimationFrame 停止。重新开始游戏时需要先清除所有动画间隔，确保小球从初始位置重新播放。

1.9.2 实验结果

成功实现完整的英雄快跑游戏，包含跳跃、坑洞生成与移动、碰撞检测、计时系统和最佳成绩存储功能。游戏可正常开始、运行和结束，最佳成绩通过 localStorage 持久化保存。

英雄快跑



当游戏失败时:



1.10 拖拽排序

1.10.1 实验过程 (包括是否遇到问题及如何解决)

本实验使用 HTML5 Drag and Drop API 实现了列表项的拖拽排序功能。核心实现包括:

- (1) dragstart 事件——记录被拖拽的元素并添加 dragging 样式类;
- (2) dragover 事件——阻止默认行为允许放置, 添加 hover 高亮样式;
- (3) drop 事件——比较拖拽项和目标项的索引, 使用 insertBefore 重新排列 DOM 元素;
- (4) dragend 事件——清理 dragging 样式;
- (5) 重置按钮——使用 originalOrder 数组重新生成列表恢复初始状态;
- (6) 辅助函数 updateOrderDisplay——遍历 DOM 获取当前顺序并更新页面显示。

遇到的问题：DOM 元素的重排需要判断拖拽项和目标项的位置关系（在上方还是下方），因为 insertBefore 的行为不同。如果拖拽项在原目标项之前，应插入到目标项之后 (nextSibling)，反之则直接插入到目标项之前。另外需要为每个动态生成的 li 重新绑定事件监听器。

1.10.2 实验结果

成功实现拖拽排序功能，用户可以拖拽列表项改变顺序，拖拽过程有高亮提示，排序结果实时显示在页面上，控制台输出'列表已更新=x,x,x,x'。重置按钮恢复初始顺序，控制台输出'列表已重置=1,2,3,4'。



1.11 通讯录管理

1.11.1 实验过程（包括是否遇到问题及如何解决）

本实验使用 IndexedDB 实现了带数据持久化的手机通讯录管理器。核心功能包括：

- (1) 初始化 IndexedDB 数据库 contactsDB，创建对象仓库 contacts，以 phone 为主键；
- (2) 实时输入校验——姓名字段使用正则 `[/^a-zA-Z\u4e00-\u9fa5\s]/g` 过滤非法字符，电话字段使用 `[/^0-9]/g` 只允许数字；
- (3) 添加联系人——检查非空、非法字符、重复号码，重复号码弹窗确认覆盖；
- (4) 删除联系人——通过 `class='delete-btn'` 的按钮触发，使用 IndexedDB 的 delete 方法删除；
- (5) 页面加载时通过游标遍历读取所有联系人数据。

遇到的问题：IndexedDB 操作全部是异步的，需要在 onsuccess 回调中处理结果。加载联系人时使用游标遍历，需要处理数据库为空的情况，显示‘通讯录为空，请添加联系人’。防止 XSS 攻击使用 `textContent` 而非 `innerHTML` 设置联系人内容。添加联系人成功后不要清空输入框以便连续添加。

1.11.2 实验结果

成功实现通讯录管理器，支持添加、删除联系人，输入实时校验，重复号码检测与覆盖确认，数据通过 IndexedDB 持久化存储。页面刷新后数据不丢失。



手机通讯录管理器



1.12 WebWorker

1.12.1 实验过程（包括是否遇到问题及如何解决）

本实验使用 Web Worker 实现质数计算，解决计算密集型任务阻塞 UI 线程的问题。实现分为以下部分：

- (1) 创建 worker.js 文件，使用 `self.onmessage` 接收主线程消息（起止范围），在 Worker 中执行质数计算，每 1000 个质数通过 `postMessage` 发送进度更新，完成后发送最终结果；
- (2) 质数判断优化——使用 `Math.sqrt(num)` 限制检查范围，从 $O(n)$ 优化到 $O(\sqrt{n})$ ；
- (3) 主线程与 Worker 通信——通过 `postMessage` 发送任务，`onmessage` 接收消息并根据 type 字段（`progress/result`）分别更新进度显示和最终结果；
- (4) 动画功能——三个蓝色圆形元素通过 `setInterval` 逐像素修改 `left` 属性实现平移动画，移动到窗口右侧时停止。

遇到的问题：动画重置 Bug——再次点击开始动画时，之前设置的 interval 仍在运行，导致小球位置异常。解决方案是使用 animationIntervals 数组存储所有 interval ID，重新开始时先清除所有 interval 并重置小球位置。Worker 中的进度通信频率需要适当，每 1000 个质数报告一次进度。

1.12.2 实验结果

成功实现 WebWorker 质数计算，计算过程不阻塞 UI 动画。主线程动画（圆形平移）和 Worker 计算并行执行，页面响应流畅。进度实时显示，计算完成后展示质数数量。动画可正确重置和重新播放。

利用 WebWorker 计算质数数量

输入范围起始： 输入范围结束：

计算完成！

范围内的质数数量是：1229

UI 动态效果展示（平移动画）



1.13 用户注册

1.13.1 实验过程（包括是否遇到问题及如何解决）

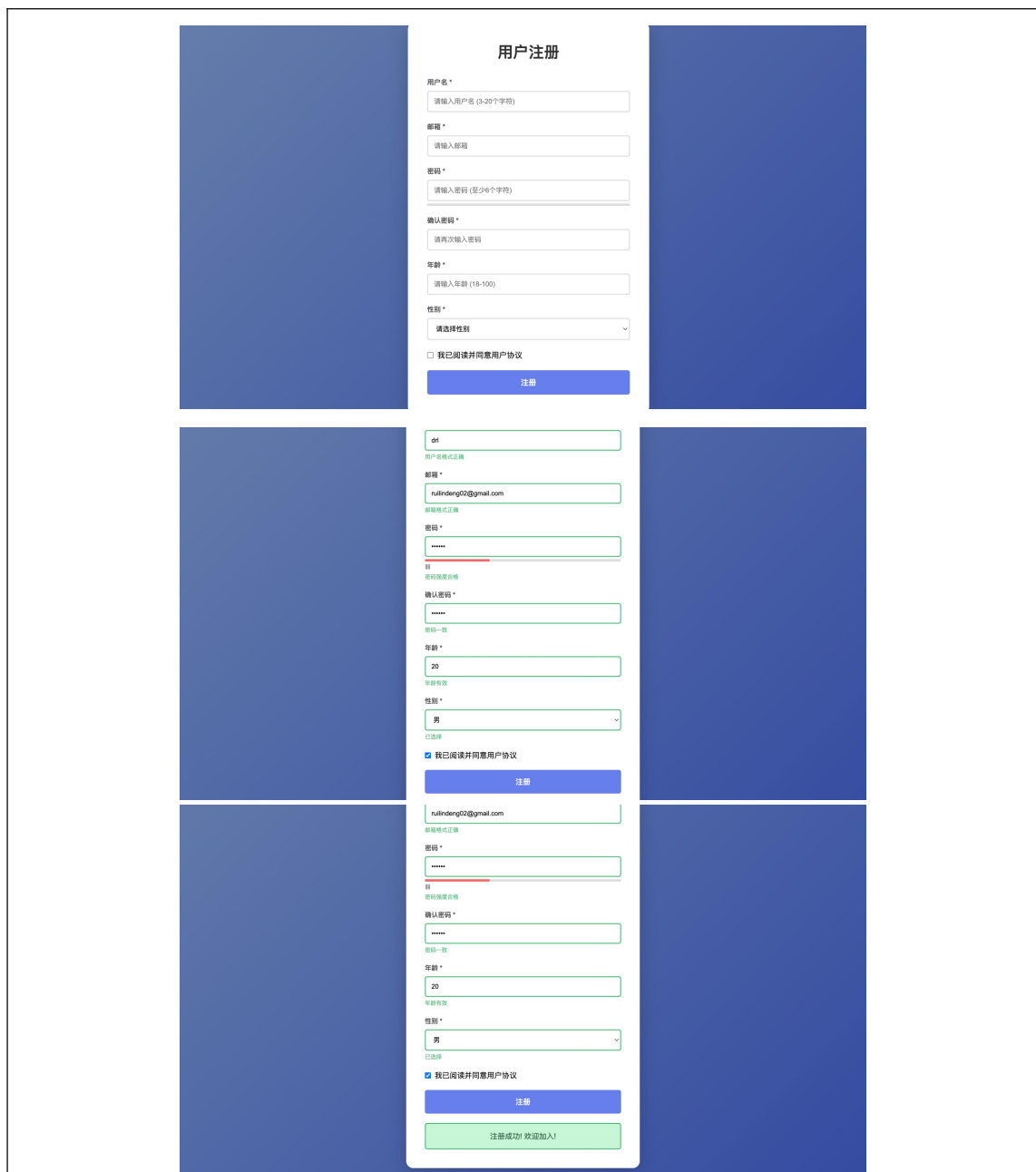
本实验实现了一个完整的用户注册表单验证系统，包含 7 项字段验证和密码强度检测。核心实现包括：

- (1) 用户名验证——非空、3-20 字符长度、正则 /^[a-zA-Z0-9_]+\$/ 限制字符；
- (2) 邮箱验证——非空、正则 /^[^s@]+@[^s@]+\.[^s@]+\$/ 验证格式；
- (3) 密码验证——非空、至少 6 字符，强度评分系统（长度 ≥ 8 、小写字母、大写字母、数字、特殊字符各 1 分），分数 ≤ 2 为弱（红色 33%），3 分为中等（橙色 66%）， ≥ 4 为强（绿色 100%）；
- (4) 确认密码验证——非空、与密码一致；
- (5) 年龄验证——非空、18-100 范围；
- (6) 性别验证——检查是否选择；
- (7) 表单提交——调用所有验证函数，检查协议勾选，全通过显示成功消息，否则显示错误消息。

遇到的问题：密码验证通过后需要重新验证确认密码（如果已有值），防止修改密码后两次输入不一致。showError/showSuccess 等辅助函数需要正确管理 input 的 error/success 类名和提示元素的 show 类。所有验证函数统一使用相同的错误/成功提示模式，便于维护。

1.13.2 实验结果

成功实现用户注册表单验证系统，所有字段的实时验证和提交验证均正常工作。密码强度条动态显示，控制台输出详细的验证日志。表单提交时进行完整验证。



2. 设计并实现某主题网站的账户模块，包括

2.1 构思、设想

账户模块是非常常见的一个模块，并且也是一个持续在改进优化用户体验，同时持续提高用户账号安全的模块。参考前述用户注册实验，并结合调研的网站，请精心设计某主题网站的账户模块相关页面，从用户体验和安全保障 2 个方面，对该模块的注册、登录等功能进行设计。

本模块还设计了配套的登录页面，与注册页面保持一致的视觉风格。登录页面支持邮箱/用户名+密码方式登录，包含"记住我"功能和忘记密码入口。为提升用户体验，还预留了微信、QQ、GitHub 等第三方快捷登录按钮的接入位置。在安全方面，登录表单同样实施了 XSS 输入过滤，对异常字符进行检测并拒绝提交。

2.2 设计实现及安全优化

针对前述构思设想情况，你是如何设计并实现该页面的。请结合“代码片段”选取用户体验和安全保障 2 个要点进行介绍。

代码片段:

代码 1: 密码强度实时评估算法 (register.html 核心逻辑)

```
// ===== 密码验证 + 强度 =====
function validatePassword() {
    var val = password.value;
    if (!val) {
        setState(password, 'password-icon', 'password-msg', 'error', '密码不能为空');
        strengthInner.className = 'strength-bar-inner';
        strengthLabel.textContent = '';
        console.log('密码验证失败: 不能为空');
        return false;
    }
    if (val.length < 8) {
        setState(password, 'password-icon', 'password-msg', 'error', '密码长度至少8个字符');
        console.log('密码验证失败: 长度不足');
        return false;
    }

    // 强度评分
    var score = 0;
    if (val.length >= 8) score++;
    if (val.length >= 12) score++;
    if (/[a-z]/.test(val)) score++;
    if (/[A-Z]/.test(val)) score++;
    if (/[\d-9]/.test(val)) score++;
    if (/["a-zA-Z0-9-]/.test(val)) score++;

    var level = '';
    if (score <= 2) {
        strengthInner.className = 'strength-bar-inner weak';
        level = '弱';
    } else if (score === 3) {
        strengthInner.className = 'strength-bar-inner medium';
        level = '中';
    } else if (score === 4) {
        strengthInner.className = 'strength-bar-inner strong';
        level = '强';
    } else {
        strengthInner.className = 'strength-bar-inner very-strong';
        level = '非常强';
    }
    strengthLabel.textContent = '密码强度: ' + level;
}
```

代码 2: XSS 防护 — textContent 安全渲染

```
function setState(input, iconId, msgId, status, text) {
    input.className = status || '';
    var icon = document.getElementById(iconId);
    var msg = document.getElementById(msgId);
    if (icon) {
        icon.className = 'status-icon' + (status ? ' show' : '');
        icon.textContent = status === 'success' ? '✓' : (status === 'error' ? '✗' : '');
    }
    if (msg) {
        msg.className = 'msg ' + (status || '');
        msg.textContent = text || '';
    }
}
```

代码 3: 登录表单 XSS 攻击特征检测

```
function validateAccount() {
    var val = account.value.trim();
    if (!val) {
        setError(account, 'account-msg', '请输入邮箱或用户名');
        console.log('账号验证失败: 不能为空');
        return false;
    }
    // XSS 检测
    if (/<script|javascript:|on\w+\s*=|/i.test(val)) {
        setError(account, 'account-msg', '包含非法字符');
        console.log('账号验证失败: 检测到XSS攻击特征');
        return false;
    }
    clearError(account, 'account-msg');
    console.log('账号验证通过');
    return true;
}
```

以上代码展示了本模块在用户体验（密码强度多维评分）和安全保障（textContent 防 XSS + 输入过滤）两个方面的核心实现策略。

基于上述构思，页面实现的核心要点如下:

用户体验实现:

(1)实时表单验证架构: 为每个输入框绑定 input 和 blur 事件, 实现即时反馈。关键代码模式为: showError()设置错误状态 (红色边框+错误文字), showSuccess()设置成功状态 (绿色边框+成功图标), hideError()/hideSuccess()清除状态。validatePassword()函数包含 5 项评分指标, 动态更新强度条 CSS 类名和宽度百分比。

(2)表单提交流程: submit 事件中调用所有验证函数, 使用标志变量汇总结果, 全部通过显示绿色成功卡片, 任一失败显示红色错误卡片。showResultMessage()函数统一管理结果消息的类型 (success/error) 和内容。

安全保障实现:

(1)多层输入过滤: 前端使用 HTML5 input 属性 (type='email'、type='password'、type='number') 提供浏览器级别的格式校验; JavaScript 正则表达式进行二次验证; 所有用户数据显示均使用textContent避免XSS。用户名正则 /^[a-zA-Z0-9_]+\$ / 严格限制字符集。

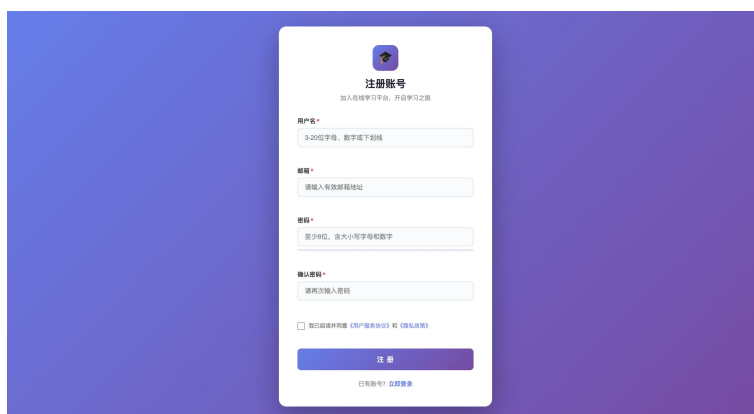
(2)密码安全策略: 最小 6 字符、强度分级评估、确认密码一致性检查。密码强度评分综合考虑长度、大小写字母、数字和特殊字符五个维度。

(3)协议确认: 必选勾选框确保用户已阅读服务条款。

2.3 效果呈现

最终的实现效果结果 (注意控制图片大小和分辨率, 1 张图不要超过 10M)

注册页面:



登录页面:



2.4 思考小结

与常见各种“用户注册、登录”对比, 你自己如何评价上述功能的设计、实现。

本账户模块的设计与实现在用户体验和安全保障方面达到了较好的平衡。

与常见用户注册登录页面对比，本实现的优势在于：

- (1) 实时验证体验优于常见的仅提交时验证方式，减少用户的挫败感；
- (2) 密码强度可视化指示器是许多主流网站（如 Google、GitHub）采用的成熟方案，能有效引导用户行为；
- (3) 清晰的颜色编码（红/绿）降低了用户的认知负担。

不足之处及改进方向：

- (1) 可增加用户名实时查重（通过 AJAX 请求后端），防止已注册用户名被重复使用；
- (2) 可增加验证码或人机验证机制防止恶意注册；
- (3) 邮箱验证邮件功能——注册后发送验证链接确保邮箱有效性；
- (4) 密码可见性切换按钮（显示/隐藏密码），方便用户检查输入；
- (5) OAuth 第三方快捷登录（如微信、QQ、GitHub），减少用户注册阻力。

总体而言，本模块较好地完成了注册功能的基础验证和安全防护，用户体验流畅，但仍需结合实际后端服务才能形成完整的账户系统。

实验结论：

1. 完成实验后，当你看到自己刚做完的账号页面，你有何感想？

完成本实验后，看到自己刚实现的账户注册页面，内心颇有成就感。从最初的数字运算、偶数平方和等基础 JS 练习，到商品清单、计数器、星级评分等 DOM 交互练习，再到英雄快跑游戏、拖拽排序、通讯录管理等复杂应用，最后汇聚到完整的用户注册表单验证系统，这一系列的实验让我系统性地掌握了 JavaScript 的核心能力。

用户注册页面虽然看似简单，但包含了表单验证、事件处理、DOM 操作、正则表达式、CSS 动画等多个知识点的综合运用。特别是密码强度实时评估、多字段联动验证等功能，体现了良好的用户体验设计思维。这些知识点不是孤立的，而是相互关联、层层递进的。

心得体会：

主要说明你对 JS 的理解和整体感受。

通过本实验的 13 个子实验和一个综合设计，我对 JavaScript 有了全面深入的理解：

第一，JS 的灵活性令人印象深刻。从基础语法（循环、函数、数组方法）到 DOM 操作，从事件处理到异步编程（IndexedDB、WebWorker），JS 展现了作为一门动态语言的强大表达能力。同样的功能可以用多种方式实现——例如数组处理可以用 for 循环，也可以用 filter/map/reduce 链式调用。

第二，浏览器 API 生态丰富。本次实验接触了 Drag and Drop API、IndexedDB、Web Worker、Canvas、localStorage 等多种 Web API，每种 API 解决了特定领域的问题。理解这些 API 的使用场景和最佳实践，是成为一名合格前端开发者的必经之路。

第三，安全意识和用户体验同样重要。在通讯录管理和用户注册实验中，需要同时考虑 XSS 防护、输入校验、错误提示等安全和体验要素。好的网页不仅要功能正确，还要对用户友好、对攻击者安全。

第四，调试和问题解决能力在实践中提升。每个实验都遇到了各种问题——DOM 索引偏移、异步回调时序、Canvas 坐标计算、拖拽排序逻辑等，通过 console.log 调试、查阅 MDN 文

档、逐步排查的方式逐一解决，积累了宝贵的实战经验。

总体而言，JS 是一门深度与广度兼备的语言。前端开发不仅仅是写界面，更涉及数据结构、算法优化、用户体验设计、安全防护等多维度知识。本次实验为今后的 Web 开发学习打下了坚实的基础。

指导教师批阅意见：

成绩评定：



指导教师签字：Basker George
2026 年 07 月 02 日